

example, I used *index.js*, making my project a JavaScript project. Opening the code in *index.js* we can see a simple 'hello world' program written in JS (Figure 2).

Writing an HTTP server is straightforward using Bun's native functions. We can write a simple one like this (Figure 3):

```
-- console.log("Hello via Bun!");
1+ Bun.serve({
2+   fetch(request) {
3+     return new
Response("Hello OSFY readers!");
4+   },
5+ })
```

To execute this, simply type:

```
bun run index.js
```

And go to <http://localhost:3000> to view the result as shown in Figure 4.

Instead of re-running the *run* command, Bun also provides an option for hot reloading, so that any changes in the file are updated instantaneously. This is achieved by:

```
bun --hot index.js
```

Benchmarking Bun and Node

Bun's selling point is speed, so I had to test it for myself. I used a simple 'hello world' program in JS as the test subject. Also, I used a hyperfine command-line tool to benchmark Bun's performance against Node's (Figure 5).

Not surprisingly, Bun ran almost five times faster than Node. Additionally, Bun's package manager installs 30 times faster than Node's npm.

So should I go for Bun?

While Bun shows considerable promise, it is not without its share of limitations and considerations. One of the primary concerns regarding Bun is its stability. As an emerging technology, it is still maturing and may exhibit occasional instability. This may deter

```
JS index.js > ...
1  - console.log("Hello via Bun!");
1+ Bun.serve({
2+   fetch(request) {
3+     return new Response("Hello OSFY readers!");
4+   },
5+ })
2  6
```

Figure 3: HTTP server

```
localhost:3000/ x +
< > ↻ localhost:3000
Hello OSFY readers!
```

Figure 4: Server response in the browser

```
sreyas@de4bian: ~/osfy-bun-test
sreyas@de4bian:~/osfy-bun-test$ hyperfine "bun index.js" "node index.js"
Benchmark 1: bun index.js
Time (mean ± σ): 21.5 ms ± 4.9 ms [User: 11.6 ms, System: 4.6 ms]
Range (min ... max): 14.8 ms ... 30.9 ms 111 runs

Benchmark 2: node index.js
Time (mean ± σ): 98.9 ms ± 11.6 ms [User: 53.0 ms, System: 26.8 ms]
Range (min ... max): 85.5 ms ... 128.5 ms 28 runs

Summary
'bun index.js' ran
4.60 ± 1.18 times faster than 'node index.js'
sreyas@de4bian:~/osfy-bun-test$
```

Figure 5: Benchmarking Bun

some developers from adopting it on a widespread basis, as they seek robust and reliable runtime environments for their projects. Also, Bun's compatibility with existing Node.js modules is not seamless. It faces hurdles in achieving full compatibility with the extensive array of modules in the Node.js ecosystem. This presents an impediment for developers who wish to transition to Bun seamlessly without encountering compatibility issues. Another evolving aspect of Bun is its support for Windows-based systems. Currently, it may not provide the same level of compatibility and performance for Windows users as it does for other

platforms. This is a consideration for developers who work primarily within the Windows environment.

Given its recent introduction to the JavaScript scene, Bun has yet to gain widespread adoption compared to more established counterparts, such as Node.js and Deno. Consequently, it may not be the first choice for applications that demand proven reliability and broad industry acceptance. However, its potential is promising. Bun is open source and you can check out its development on its GitHub repository at <https://github.com/oven-sh/bun>. **END** 🐧

By: S. Sreyas

The author is a skilled programmer who has gained expertise in the field through self-learning. He has made significant contributions to various open source projects, with a particular focus on reverse engineering, compiler design, and system level programming. His contributions are available on GitHub.